

README

Helix Cluster OS

Helix Cluster OS is a next-generation distributed operating system for orchestrating compute workloads across heterogeneous nodes — from datacenter GPUs down to edge SBCs and handhelds. It unifies HPC scheduling, container orchestration, AI/ML inference, federated multi-cluster operation, and secure multi-tenant session management under a single control plane.

Engineering guarantee (CLAUDE-1 / CLAUDE-2): every feature ships with tests that prove real end-user behaviour — never green tests over stubs — and every OS-specific capability uses a real native facility per platform (no Linux-only mocks). See [CLAUDE.md](#) and [Constitution.md](#).

Features

- **Heterogeneous Node Management** — Register, monitor, and schedule across tiers T1–T8 (datacenter → microcontroller) with structured health scoring and capability negotiation.
- **Omega-model Scheduler** — Pluggable placement with optimistic concurrency, ClassAd matching, gang scheduling, value-multiplier preemption, multifactor (age/fairshare/size/QoS) priority queues, and constraint-based (location/colocation/order/stickiness) placement.
- **Distributed-Systems Foundation** — A large pure-Go library (pkg/) of consensus, membership (SWIM gossip), replication (CRDT, MVCC, anti-entropy), and federation primitives. See [Foundation Packages](#).
- **Federation & Multi-cluster** — Cross-cell trust, topology patterns, CRDT config sync, data-residency admission, split-brain detection, and quorum-based failure confirmation.
- **GPU & Cost Orchestration** — GPU pool management, TCO-aware local cost modelling, cost/latency-aware schedulers, burst-to-cloud autoscaling, N+K failover capacity reserve, and global budget caps.
- **Security & Attestation** — SPIFFE identity, JWT auth, ML-KEM-768 post-quantum E2EE, device attestation (challenge/response, proof-of-GPU-work, device sealing), and attestation-gated admission.
- **Deterministic Simulation Testing (DST)** — FoundationDB-style seeded simulation, BUGGIFY fault injection, Turmoil network simulation, and clock-fault injectors for reproducible distributed-systems testing.
- **Observability** — Built-in metrics, W3C distributed tracing, structured logging, and Grafana dashboard generation.
- **Multi-Protocol APIs** — gRPC services with Protocol Buffer definitions for all subsystems.

Architecture

Helix is organised as a seven-layer stack (L0–L7) with 14 control-plane microservices, coordinated via SWIM gossip for membership and Raft consensus for strongly-consistent state. The scheduler is an Omega-model two-level design with optimistic concurrency.

The repository is a Go workspace combining a core module with git submodules for the larger services.

```
.
├─ api/v1/          # Protocol Buffer definitions (NodeService,
SessionService, SchedulerService, ...)
├─ cmd/             # Service binaries and CLIs
├─ internal/        # Private application packages (console,
gateway, scheduler, node, health, policy, trust, ...)
├─ pkg/             # Shared pure-Go foundation library (see
"Foundation Packages")
├─ web/             # React + TypeScript + Vite dashboard
├─ docs/            # Documentation (see "Documentation")
├─ data/            # HXC registry (SQLite) and runtime data
├─ scripts/         # Build, test, and utility scripts
├─ HelixConstitution/ # Governance & constitution (submodule)
├─ security/        # Security service: identity, E2EE,
attestation (submodule)
├─ helixqa/         # HelixQA challenge/validation framework
(submodule)
├─ EventBus/        # Event bus (submodule)
├─ Messaging/        # Messaging service (submodule)
├─ discovery/        # Service discovery (submodule; pkg/
discovery is the editable root)
├─ containers/      # Container runtime (submodule)
├─ recovery/        # Recovery service (submodule)
├─ config/          # Configuration service (submodule)
├─ challenges/      # Challenge platform (submodule)
├─ DocProcessor/    # Document processing (submodule)
├─ LLMOrchestrator/ # LLM orchestration (submodule)
├─ Herald/          # Notification service (submodule)
├─ docs_chain/      # Documentation-chain engine (submodule)
```

Foundation Packages

The pkg/ library provides the pure-Go, deterministic, well-tested primitives the control plane is built from. Highlights by domain:

Domain	Packages
Consensus & coordination	voting (largest-subcluster quorum), failconfirm (SWIM two-phase PFAIL→FAIL), leader, lock,

Domain	Packages
Membership & discovery	splitbrain / splitbrainalert, heartbeatcoalescer (Multi-Raft) swim (+ phi-accrual, hierarchical), discovery (+ federated), nattraversal (STUN), ice, cellmesh
Replication & state	crdt (+ merkle, LWW/ORSet/G/PN-counters/vector-clock), mvcc (B-tree time-travel store), antientropy (hinted-handoff + read-repair + Merkle diff), watchmanager (syncd/unsyncd/victim), hlc, offlinesync, checkpoint_merge
Scheduling & placement	scheduler (Omega/ClassAd/gang/preempt), constraints (Pacemaker 4-type), preempt (value-multiplier), priorityqueue (multifactor aging), backfill (SLURM), admissioncontrol (N+K reserve), budgetcap, qos, suitability, ewmarank, workclaim (SKIP LOCKED), providerchain (multi-tier fallback cascade), modelrouter (strategy→default model), gepetto (local-vs-Chutes arbitration)
GPU & resource mgmt	pool, local (TCO), costsched, latencysched, healthmonitor, gpuattest (attestation crypto), capability, deviceprofile, device, tierdef, tiersec, quantization, gpucatalog (compute-multiplier catalog + attested scoring)
Federation & multi-cluster	federation (+ suspicion), fedtopology, fedtrust, configsync, residency, raftprofile, spiffefed, doublecrypt
Messaging & flow	flowcontrol (K8s APF), workqueue (rate-limited), ratelimit, backoff, retry, idempotent (exactly-once), rebalance (cooperative-sticky), fiber, fallbackchain, pubsub, events
Routing & sessions	hashslot (CRC16 + MOVED/ASK), session, slotmigration (atomic live migration), edge, edgeregistry, edgeverify, edgefusion
Security & verification	crypto, jwt, modelintegrity (SHA-256 gate), redundanttexec

Domain	Packages
	(BOINC trust), attestadmit, doublecrypt, spiffefed, gravaladmit (HMAC GraVal admission), gravalverify (VRAM-ratio attestation + BatchVerify KPI)
Burst & economics	burst (hysteresis autoscaler), bursthysteresis (MONITOR→SPILL→RECOVER dead-band), cloudspot, marketplace, chutesaccount (Chutes API model-list + balance client)
Testing & simulation	testing/dst (+ BUGGIFY, chaos, turmoil), timefault, chaosexp, fmea, phasegate, qualitygate, phase7matrix, stats, covgate, sandbox
Observability	metrics, tracing (W3C), health, grafanadash, log

Each package is standard-library-only where possible, deterministic (injected clocks, seeded PRNGs), and proven by tests that fail under mutation of the logic they cover.

Quick Start

Prerequisites

- Go 1.26+ (workspace uses go.work)
- Node.js 20+ (for the web UI)
- SQLite 3 (the HXC registry lives at data/hxc_registry.db)
- Docker & Docker Compose (for integration services)
- Protocol Buffer compiler + protoc-gen-go (optional, for API regeneration)

Setup / Build / Test

```
./scripts/setup.sh      # initialise submodules and toolchains
./scripts/build.sh      # go build ./...
./scripts/test.sh       # go test ./...
./scripts/lint.sh       # go vet + linters
./scripts/format.sh     # gofmt
```

To run the full race suite for a package:

```
go test -race -count=1 ./pkg/<package>/...
```

API

Protocol Buffer definitions live in `api/v1/`. Core services:

- `NodeService` — Node lifecycle management
- `SessionService` — Session CRUD operations
- `SchedulerService` — Job scheduling and monitoring
- `HealthService` — Health checks and reporting
- `AdvisoryService` — Distributed locks and advisory events
- `SecurityService` — Authentication and authorization
- `BuildService` — Build pipeline management

Web UI

The web dashboard is built with React, TypeScript, and Vite.

```
cd web && npm install && npm run dev
```

Documentation

- [CLAUDE.md](#) — AI-agent engineering rules (end-user usability & cross-platform parity guarantees)
- [Constitution.md](#) / [HelixConstitution/](#) — project governance
- [docs/FOUNDATION_PACKAGES.md](#) — full catalogue of pkg/ packages
- [docs/HXC_REGISTRY.md](#) — the work-item registry model
- [CODING_STANDARDS_G0.md](#) / [CODING_STANDARDS_C.md](#) / [CODING_STANDARDS_ZIG.md](#) — language standards
- [DEVELOPMENT.md](#) — development workflow
- [CHANGELOG.md](#) — release history

Documentation Synchronization (CLAUDE-3 / §11.4.106)

Every change to code, services, components, architecture, or schema MUST update all affected materials — README, docs, user guides, manuals, websites, diagrams, and SQL/schema definitions — together with **all** of their exports. This is mechanically enforced out-of-the-box by the `docs_chain` engine via `.docs_chain/contexts/*.yaml` (Markdown → HTML/PDF/DOCX) and gated by `docs_chain verify`, with no escape hatch. The mandate restates and cites Constitution §11.4.106 and the project rules CLAUDE-3 / AGENT-2 / QWEN-2.

Development Model

Work is tracked in an SQLite registry (`data/hxc_registry.db`) of HXC-#### items across 11 phases (0–10). Items move Queued → In progress → Completed and are implemented in parallel “waves”: disjoint new packages built via an implement → adversarial-review → fix pipeline, then gated with whole-tree build/vet, `-race` tests, and an independent mutation bite per item that must fail the item’s named guard test. No item is marked Completed without that proof.

Contributing

1. Create a feature branch
2. Implement with tests that prove real behaviour (and fail under mutation)
3. Run `go build ./... && go vet ./... && go test -race ./...`
4. Commit and open a Pull Request

License

See [LICENSE](#) for details.